

Docker Namespace and Cgroups



Kasun Rathnayaka · [Follow](#)

4 min read · Feb 19, 2018



Listen



Share



More

Namespaces are one of a feature in the Linux Kernel and fundamental aspect of containers on Linux. On the other hand, namespaces provide a layer of isolation. Docker uses namespaces of various kinds to provide the isolation that containers need in order to remain portable and refrain from affecting the remainder of the host system. Each aspect of a container runs in a separate namespace and its access is limited to that namespace.

Namespace Types:

1. Process ID
2. Mount
3. IPC (Interprocess communication)
4. User (currently experimental support for)
5. Network

In my knowledge, the most effective way to prevent privilege-escalation attacks from within a container is to configure your container's applications to run as unprivileged users. For containers whose processes must run as the **root** user within the container, you can re-map this user to a less-privileged user on the Docker host. The mapped user is assigned a range of UIDs (0- 65536), but have no privileges on the host machine itself.

when we run docker with root use that easy to install packages and manage containers. Unfortunately, the issue is within docker container users can access the underline host machine as below. That's a big risk ...

```
# Run a container and mount host /etc onto /root/etc
$ docker run --rm -v /etc:/root/etc -it ubuntu

# Make some change on /root/etc/hosts
root@29373h1f1332:/# vi /root/etc/hosts

# Exit from the container
root@29373h1f1332:/# exit

# Check /etc/hosts and try to delete it
$ cat /etc/hosts
$ rm /etc/hosts
```

As you can see, it is surprisingly easy, and it's obvious that Docker wasn't designed for shared computers. But now, with the User Namespaces, Docker lets you avoid this problem.

Enabling User Namespaces

Mainly Namespace Remapping manages two files:

1. /etc/subuid
2. /etc/subgid

You can follow the below example to understand how to setup namespace in a docker environment.

```
# Create a user called "dockremap"
$ sudo adduser dockremapper

# Setup subuid and subgid
$ sudo sh -c 'echo dockremap:400000:65536 > /etc/subuid'
$ sudo sh -c 'echo dockremap:400000:65536 > /etc/subgid'

$ kasunr@kasunr-desktop:~$ vim /etc/docker/daemon.json

{
  "userns-remap": "dockremap"
}
```

And restart Docker:

```
$ sudo /etc/init.d/docker restart
```

Then you could see new docker containers couldn't have permission to the host machine /etc/

```
# Run a container and mount host's /etc onto /root/etc
root@kasunr-desktop:~# docker run --rm -v /etc:/root/etc -it ubuntu

# try to delete host machine files
$ root@88373h1f1332:~# rm /root/etc/hosts
rm: cannot remove 'hosts': Permission denied
```

check the permission and file hierarchic new docker setup

```
root@kasunr-desktop:/var/lib/docker/400000.400000# cd ../
root@kasunr-desktop:/var/lib/docker# ls
/var/lib/docker/400000.400000/
aufs builder containerd containers image network plugins
runtimes swarm tmp trust volumes
root@kasunr-desktop:/var/lib/docker# ls
/var/lib/docker/400000.400000/ -l
total 48
drwx----- 5 400000 400000 4096 Feb 16 12:09 aufs
drwx----- 2 root    root    4096 Feb 16 12:09 builder
drwx--x--x  3 root    root    4096 Feb 16 12:09 containerd
drwx----- 3 400000 400000 4096 Feb 16 15:06 containers
drwx----- 3 root    root    4096 Feb 16 12:09 image
drwxr-x---  3 root    root    4096 Feb 16 12:09 network
drwx----- 4 root    root    4096 Feb 16 12:09 plugins
drwx----- 2 root    root    4096 Feb 16 12:09 runtimes
drwx----- 2 root    root    4096 Feb 16 12:09 swarm
drwx----- 2 400000 400000 4096 Feb 16 12:10 tmp
drwx----- 2 root    root    4096 Feb 16 12:09 trust
drwx----- 2 400000 400000 4096 Feb 16 12:09 volumes
```

To disable user namespaces for a specific container, add the `--userns=host` flag to the docker container create, docker container run, or docker container exec command.

More about linux namespace : <https://www.linux.com/news/understanding-and-securing-linux-namespaces>

Cgroups provide resource limitation and reporting capability within the container space. They allow granular control over what host resources are allocated to the containers and when they are allocated.

Common control groups

1. CPU
2. Memory
3. Network Bandwidth
4. Disk
5. Priority

we could have a lot of flexibility to manage every cgroup according to our requirements.

Note : More information: [Cgroups](#)

For example you could see `systemd-cgls` and `systemd-cgtop` commands show how cgroup working on the underline host server.

```
kasunr@~:~ $ systemd-cgls
```

Control Group	Tasks	%CPU	Memory	Input/s	Output/s
/	-	1.6	649.4M	-	-
/user.slice	12	1.3	27.3M	-	-
/system.slice	50	0.3	604.0M	-	-
/system.slice/docker.service	17	0.3	233.0M	-	-
/system.slice/iscsid.service	2	0.0	2.0M	-	-
/init.scope	1	-	2.3M	-	-
/system.slice/accounts-daemon.service	3	-	3.8M	-	-
/system.slice/acpid.service	1	-	204.0K	-	-
/system.slice/atd.service	1	-	244.0K	-	-
/system.slice/boot.mount	-	-	40.0K	-	-
/system.slice/cron.service	1	-	368.0K	-	-
/system.slice/dbus.service	1	-	1.9M	-	-
/system.slice/dev-mapper-server\x2d\x2dvg\x2dswap_1.swap	-	-	188.0K	-	-
/system.slice/lvm2-lvmetad.service	1	-	324.0K	-	-
/system.slice/lxcfs.service	3	-	648.0K	-	-
/system.slice/mdadm.service	1	-	684.0K	-	-
/system.slice/polkitd.service	3	-	1.0M	-	-
/system.slice/rsyslog.service	4	-	1.7M	-	-
/system.slice/snopd.service	6	-	27.7M	-	-
/system.slice/ssh.service	1	-	5.8M	-	-
/system.slice/sys-kernel-debug.mount	-	-	876.0K	-	-
/system.slice/system-getty.slice	-	-	3.5M	-	-
/system.slice/system-systemd\x2dfsck.slice	-	-	916.0K	-	-
/system.slice/systemd-journald.service	1	-	3.9M	-	-
/system.slice/systemd-logind.service	1	-	988.0K	-	-
/system.slice/systemd-timesyncd.service	2	-	560.0K	-	-
/system.slice/systemd-udev.service	1	-	26.7M	-	-
/user.slice/user-1000.slice	12	-	-	-	-

Control group list

Using these cgroup policies is very simple. If you for instance want to lock down a Docker container to the first CPU core, you'd append `--cpuset-cpus=0` to your docker

run command. And also you could setup the cpu shares which you required.

```
$ docker run -d --name='kasun_priority_1' --cpuset-cpus=0 --cpu-shares=20  
kasuntest1
```

```
$ docker run -d --name='kasun_priority_2' --cpuset-cpus=2 --cpu-shares=30  
kasuntest2
```

And we could use the below command to change running container CPU shares.

```
sudo systemctl set-property docker-  
7d7ds9s7dhdsda044b29cb873cac460b429cfcbbdd0e877c0868eb2a901dbf80.scope  
CPUShares=512
```

In Summary, **Namespace** gives the isolation for the container with the underline host where **Cgroup** gives the ability to allocate things to those containers. In my point of view, we would get some understanding and practical experience with namespace and Cgroups in the blog. I will share my more experience with the next post !.

Docker

Cgroups

DevOps

Containers



Follow



Written by Kasun Rathnayaka

229 Followers

DevOps/Cloud Solution Architect , DevSecOps Practices Lead
